Шпаргалка

pandas, matplotlib и scikit-learn

для анализа данных и задач классификации

Справочник методов и примеров кода: загрузка и очистка данных, визуализация, предобработка, алгоритмы классификации, метрики, кросс-валидация, подбор гиперпараметров, работа с дисбалансом классов, интерпретация моделей. Код с цветовой подсветкой синтаксиса и подписями к каждой строке: что и за что отвечает. Содержание кликабельно, в панели закладок PDF-просмотрщика есть навигация по разделам.

Содержание

1. Pandas 3

1.1 Загрузка и сохранение 3

1.2 Первичный осмотр данных (EDA) 3

1.3 Выбор данных 3

1.4 Пропуски 4

1.5 Дубликаты, типы, переименование 4

1.6 Преобразования и создание признаков 4

1.7 Строки и даты 4

1.8 Группировка и агрегации 5

1.9 Объединение таблиц 5

1.10 Кодирование категорий 5

1.11 Выбросы 5

1.12 Разное 6

2. Matplotlib 7

2.1 Основы 7

2.2 Типы графиков 7

2.3 Несколько графиков 8

2.4 Графики прямо из pandas 8

2.5 Визуализации для EDA и ML 8

3. Scikit-learn для классификации 10

3.1 Подготовка данных и разбиение . . . 10
3.2 Предобработка . . . 10
3.3 Алгоритмы классификации . . . 12
3.4 Когда что выбирать . . . 15
3.5 Обучение и предсказание . . . 15
3.6 Метрики классификации . . . 15
3.7 Кросс-валидация . . . 16
3.8 Подбор гиперпараметров . . . 17
3.9 Дисбаланс классов . . . 17
3.10 Калибровка вероятностей . . . 18
3.11 Отбор и снижение размерности признаков . . . 18
3.12 Интерпретация моделей . . . 19
3.13 Текстовая классификация . . . 19
3.14 Сохранение и загрузка модели . . . 19
3.15 Вспомогательные утилиты . . . 19

4. Полный пример: от данных до модели . . . 20

5. Частые ошибки . . . 21

1. Pandas

Pandas работает с таблицами: **DataFrame** (таблица), **Series** (один столбец), **Index** (метки строк). Ниже у каждой строки кода есть подпись, что она делает.

1.1 Загрузка и сохранение

```
import pandas as pd  # главная библиотека для работы с таблицами
import numpy as np   # числовые массивы, NaN, математические функции

df = pd.read_csv(                               # чтение CSV-файла в таблицу df
    "data.csv",                                 # путь к файлу
    sep=",",                                    # разделитель столбцов (";" или "\t" для других форматов)
    encoding="utf-8",                           # кодировка (для старых русских файлов "cp1251")
    index_col=None,                             # столбец, который станет индексом строк
    usecols=["a", "b"],                         # читать только эти столбцы (экономит память)
    dtype={"a": "int32"},                       # явно задать типы столбцов
    parse_dates=["date"],                       # сразу превратить столбец в даты
    na_values=["?", "NA"],                      # какие значения считать пропусками (NaN)
    nrows=1000,                                 # прочитать только первые 1000 строк
)
df = pd.read_excel("data.xlsx", sheet_name=0)   # Excel, лист по номеру или имени
df = pd.read_json("data.json")                  # JSON-файл
df = pd.read_parquet("data.parquet")            # Parquet: быстрый и компактный формат
df = pd.read_sql("SELECT * FROM t", connection) # результат SQL-запроса

df.to_csv("out.csv", index=False)     # сохранить в CSV без столбца индекса
df.to_parquet("out.parquet")          # сохранить в Parquet
df.to_excel("out.xlsx", index=False)  # сохранить в Excel
```

1.2 Первичный осмотр данных (EDA)

```
df.head()                                       # первые 5 строк
df.tail()                                       # последние 5 строк
df.sample(5)                                    # 5 случайных строк
df.shape                                        # (число строк, число столбцов)
df.columns                                      # названия столбцов
df.dtypes                                       # тип данных каждого столбца
df.index                                        # индекс строк
df.info()                                       # типы, число непустых значений, память
df.describe()                                   # статистики числовых столбцов: среднее, std, квартили
df.describe(include="object")                   # статистики текстовых столбцов
df.describe(include="all")                      # статистики по всем столбцам сразу
df.nunique()                                    # число уникальных значений в каждом столбце
df["col"].value_counts(normalize=True)          # доли каждого значения в столбце
df["col"].unique()                              # список уникальных значений
df.memory_usage(deep=True)                      # сколько памяти занимает каждый столбец
df.isna().sum()                                 # число пропусков в каждом столбце
df.isna().mean() * 100                          # процент пропусков в каждом столбце
df.duplicated().sum()                           # число полностью повторяющихся строк
df.corr(numeric_only=True)                      # корреляция Пирсона между числовыми столбцами
df.corr(method="spearman", numeric_only=True)   # ранговая корреляция (устойчивее к выбросам)
df.skew(numeric_only=True)                      # асимметрия распределения (0 = симметрично)
df.kurt(numeric_only=True)                      # эксцесс: «тяжесть хвостов» распределения
```

1.3 Выбор данных

```
df["a"]                                 # один столбец (Series)
df[["a", "b"]]                          # несколько столбцов (DataFrame)
df.loc[0:5, ["a", "b"]]                 # выбор по меткам: строки 0..5 ВКЛЮЧИТЕЛЬНО
df.iloc[0:5, 0:2]                       # выбор по позициям: строки 0..4, столбцы 0..1
df[df["a"] > 10]                        # фильтр строк по условию
df[(df["a"] > 10) & (df["b"] == "x")]   # несколько условий: & = И, | = ИЛИ, ~ = НЕ
df[df["b"].isin(["x", "y"])]            # значение входит в список
df[df["b"].between(1, 5)]               # значение в диапазоне [1, 5]
df.query("a > 10 and b == 'x'")         # условие в виде строки (читается проще)
df.select_dtypes(include="number")      # только числовые столбцы
df.select_dtypes(exclude="number")      # все, кроме числовых
df.filter(like="feature_")              # столбцы, в названии которых есть подстрока
df.filter(regex="^f_")                  # столбцы, подходящие под регулярное выражение
df.nlargest(5, "a")                     # 5 строк с наибольшими значениями a
df.nsmallest(5, "a")                    # 5 строк с наименьшими значениями a
df.at[0, "a"]                           # быстрый доступ к одной ячейке по меткам
df.iat[0, 1]                            # быстрый доступ к одной ячейке по позициям
```

1.4 Пропуски

```
df.dropna()                                    # удалить строки, где есть хотя бы один NaN
df.dropna(subset=["a"], how="any")             # смотреть только столбец a
df.dropna(thresh=3)                            # оставить строки минимум с 3 непустыми значениями
df["a"].fillna(df["a"].median())               # заполнить медианой (устойчива к выбросам)
df["b"].fillna(df["b"].mode()[0])              # заполнить самым частым значением
df.ffill()                                     # протянуть предыдущее значение вниз
df.bfill()                                     # протянуть следующее значение вверх
df["a"].interpolate(method="linear")           # линейная интерполяция (для временных рядов)
df.fillna({"a": 0, "b": "unknown"})            # свои значения для каждого столбца
df["a_missing"] = df["a"].isna().astype(int)   # признак «значение было пропущено» (0/1)
```

1.5 Дубликаты, типы, переименование

```
df.drop_duplicates(subset=["a"], keep="first")             # удалить дубли по a, оставить первый
df["a"] = df["a"].astype("float32")                        # сменить тип (float32 вдвое легче float64)
df["cat"] = df["cat"].astype("category")                   # категориальный тип: экономит память
df["d"] = pd.to_datetime(df["d"], format="%Y-%m-%d", errors="coerce")  # строка -> дата
df["n"] = pd.to_numeric(df["n"], errors="coerce")          # строка -> число (ошибки станут NaN)
df.rename(columns={"old": "new"})                          # переименовать столбец
df.columns = df.columns.str.lower().str.replace(" ", "_")  # привести имена к единому виду
df.drop(columns=["x"])                                     # удалить столбец
df.drop(index=[0, 1])                                      # удалить строки по индексу
df.reset_index(drop=True)                                  # сбросить индекс на 0, 1, 2...
df.set_index("id")                                         # сделать столбец индексом
df.sort_values(["a", "b"], ascending=[True, False])        # сортировка по двум столбцам
df.sort_index()                                            # сортировка по индексу
df.replace({"a": {0: np.nan}})                             # заменить нули в столбце a на NaN
```

1.6 Преобразования и создание признаков

```
df["new"] = df["a"] * 2                                  # новый столбец из арифметики над столбцами
df["new"] = df.apply(lambda r: r["a"] + r["b"], axis=1)  # функция к каждой строке (медленно)
df["a"].map({"x": 1, "y": 2})                            # заменить значения по словарю
df["a"].apply(np.log1p)                                  # применить функцию к каждому элементу (log(1+x))
df["bin"] = pd.cut(                                      # разбить числа на интервалы заданной ширины
    df["age"], bins=[0, 18, 35, 60, 100],                # границы интервалов
    labels=["child", "young", "adult", "senior"],        # названия интервалов
)
df["qbin"] = pd.qcut(df["income"], q=4)                  # интервалы с равным числом строк (квартили)
df["rank"] = df["a"].rank(method="dense")                # ранг значения (1, 2, 3...)
df["a"].clip(lower=0, upper=100)                         # обрезать значения по границам
df["a"].where(df["a"] > 0, 0)                            # где условие ложно, подставить 0
np.where(df["a"] > 0, 1, 0)                              # быстрый if-else по всему столбцу
df.assign(c=lambda x: x.a + x.b)                         # добавить столбец в цепочке вызовов
df["a"].shift(1)                                         # сдвиг на 1 строку (лаг, предыдущее значение)
df["a"].diff()                                           # разность с предыдущей строкой
df["a"].pct_change()                                     # относительное изменение в процентах
df["a"].rolling(7).mean()                                # скользящее среднее по окну в 7 строк
df["a"].expanding().sum()                                # накопительная сумма по растущему окну
df["a"].ewm(span=7).mean()                               # экспоненциальное скользящее среднее
df["a"].cumsum()                                         # кумулятивная сумма
df["a"].cumprod()                                        # кумулятивное произведение
```

1.7 Строки и даты

```
s = df["text"].str                          # доступ к строковым методам столбца
s.lower()                                   # в нижний регистр
s.upper()                                   # в верхний регистр
s.strip()                                   # убрать пробелы по краям
s.len()                                     # длина строки
s.contains("abc", case=False, na=False)     # содержит подстроку (без учёта регистра)
s.replace(r"\d+", "", regex=True)           # замена по регулярке (тут: убрать цифры)
s.extract(r"(\d+)")                         # достать из текста первое число
s.split(" ", expand=True)                   # разбить по пробелу на отдельные столбцы
s.startswith("a")                           # начинается с «a»
s.endswith("z")                             # заканчивается на «z»
s.get_dummies(sep="|")                      # one-hot по значениям, разделённым "|"

dt = df["date"].dt                                  # доступ к методам дат (столбец должен быть datetime)
dt.year                                             # год
dt.month                                            # месяц
dt.day                                              # день месяца
dt.dayofweek                                        # день недели: 0 = понедельник
```

```
dt.hour                                          # час
dt.quarter                                       # квартал
dt.is_month_end                                  # последний ли это день месяца
dt.day_name()                                    # название дня недели
(df["d2"] - df["d1"]).dt.days                    # разница двух дат в днях
df.set_index("date").resample("M").sum()         # агрегация по месяцам
pd.date_range("2024-01-01", periods=10, freq="D")  # последовательность дат
```

1.8 Группировка и агрегации

```
df.groupby("cat")["a"].mean()                          # среднее a по каждой категории
df.groupby(["c1", "c2"]).agg(                          # несколько агрегаций сразу
    mean_a=("a", "mean"),                              # новое_имя=(столбец, функция)
    max_b=("b", "max"),                                # максимум b в группе
    cnt=("a", "count"),                                # число непустых значений a
).reset_index()                                        # вернуть ключи группировки в обычные столбцы
df.groupby("cat")["a"].transform("mean")               # значение группы для КАЖДОЙ строки
df["a_norm"] = df.groupby("cat")["a"].transform(       # стандартизация внутри группы
    lambda x: (x - x.mean()) / x.std())
df.groupby("cat").apply(lambda g: g.nlargest(2, "a"))  # топ-2 строки в каждой группе
df.groupby("cat")["a"].agg(["mean", "std", "min", "max"])  # набор статистик
df.groupby("cat").size()                               # число строк в каждой группе
df.groupby("cat").filter(lambda g: len(g) > 5)         # оставить группы больше 5 строк
df.groupby("cat").cumcount()                           # номер строки внутри группы

pd.pivot_table(df, values="a", index="r", columns="c",  # сводная таблица
               aggfunc="mean", fill_value=0)           # функция агрегации, чем заполнить пустоты
pd.crosstab(df["a"], df["b"], normalize="index")       # таблица сопряжённости (доли по строкам)
df.melt(id_vars=["id"], value_vars=["x", "y"],         # из широкой таблицы в длинную
        var_name="var", value_name="val")              # имена новых столбцов
df.stack()                                             # столбцы -> уровень индекса (в длинный вид)
df.unstack()                                           # обратно: уровень индекса -> столбцы
```

1.9 Объединение таблиц

```
pd.concat([df1, df2], axis=0, ignore_index=True)  # склеить таблицы друг под другом
pd.concat([df1, df2], axis=1)                     # склеить таблицы рядом (по индексу)
pd.merge(df1, df2, on="id", how="left")           # SQL JOIN: inner / left / right / outer
pd.merge(df1, df2, left_on="a", right_on="b",     # ключи называются по-разному
         suffixes=("_l", "_r"))                   # суффиксы для одинаковых имён столбцов
df1.join(df2, how="left", rsuffix="_r")           # соединение по индексу
pd.merge_asof(left, right, on="time")             # соединение по ближайшему времени
```

1.10 Кодирование категорий

```
pd.get_dummies(df, columns=["cat"], drop_first=True, dtype=int)  # one-hot кодирование
df["cat_code"] = df["cat"].astype("category").cat.codes          # label encoding: категория -> номер
df["freq"] = df["cat"].map(                                      # frequency encoding: категория -> её
                                                                 # частота
    df["cat"].value_counts(normalize=True))
# target encoding (категория -> среднее таргета): считать ТОЛЬКО на train, иначе утечка
```

1.11 Выбросы

```
Q1, Q3 = df["a"].quantile([0.25, 0.75])                # 1-й и 3-й квартили
IQR = Q3 - Q1                                          # межквартильный размах
mask = df["a"].between(Q1 - 1.5 * IQR, Q3 + 1.5 * IQR)  # правило 1.5*IQR
df_clean = df[mask]                                    # строки без выбросов

z = (df["a"] - df["a"].mean()) / df["a"].std()  # z-оценка: сколько сигм от среднего
df[np.abs(z) < 3]                               # правило трёх сигм

df["a"] = df["a"].clip(                                # винзоризация: обрезать хвосты
    df["a"].quantile(0.01), df["a"].quantile(0.99))  # по 1-му и 99-му перцентилям
```

1.12 Разное

```
df.sample(frac=0.8, random_state=42)                    # случайные 80% строк (воспроизводимо)
df.copy(deep=True)                                      # независимая копия таблицы
df.pipe(func)                                           # передать df в функцию func (удобно в цепочках)
df.to_numpy()                                           # преобразовать в массив NumPy
df.style.background_gradient()                          # цветовая раскраска в Jupyter
pd.set_option("display.max_columns", None)              # показывать все столбцы при печати
pd.set_option("display.float_format", "{:.3f}".format)  # формат вывода чисел
df.eval("c = a + b")                                    # быстрое вычисление выражения
df.explode("list_col")                                  # каждый элемент списка в отдельную строку
df.T                                                    # транспонирование (строки <-> столбцы)
df.apply(pd.Series.nunique)                             # функция к каждому столбцу
df.agg(["mean", "median"])                              # несколько статистик по всем столбцам
df.memory_usage().sum()                                 # общий объём памяти в байтах
```

2. Matplotlib

Основные объекты: **Figure** (всё полотно) и **Axes** (отдельный график на нём). Привычный путь: `fig, ax = plt.subplots()`, затем рисуем методами `ax.*`.

2.1 Основы

```python
import matplotlib.pyplot as plt  # интерфейс для построения графиков

fig, ax = plt.subplots(figsize=(8, 5), dpi=100)        # полотно 8x5 дюймов и один график
ax.plot(                                               # линейный график
    x, y,                                              # данные по осям
    color="tab:blue",                                  # цвет линии
    linestyle="--",                                    # стиль: "-" сплошная, "--" штрих, ":" точки
    marker="o",                                        # маркер в точках данных
    label="line",                                      # подпись для легенды
)
ax.set_title("Title")                                  # заголовок графика
ax.set_xlabel("X")                                     # подпись оси X
ax.set_ylabel("Y")                                     # подпись оси Y
ax.set_xlim(0, 10)                                     # диапазон оси X
ax.set_ylim(0, 1)                                      # диапазон оси Y
ax.set_xscale("log")                                   # логарифмическая шкала
ax.grid(alpha=0.3)                                     # сетка (alpha = прозрачность)
ax.legend(loc="best")                                  # легенда в оптимальном месте
ax.annotate("peak", xy=(2, 5), xytext=(3, 6),          # текст с указателем: куда, где текст
            arrowprops=dict(arrowstyle="->"))          # стрелка от текста к точке
ax.axhline(0.5, color="r")                             # горизонтальная линия на уровне y=0.5
ax.axvline(3, color="g")                               # вертикальная линия на x=3
ax.fill_between(x, y1, y2, alpha=0.2)                  # залить область между двумя кривыми
ax.tick_params(axis="x", rotation=45)                  # повернуть подписи делений оси X
plt.tight_layout()                                     # убрать наложение подписей и заголовков
plt.savefig("plot.png", dpi=300, bbox_inches="tight")  # сохранить в файл
plt.show()                                             # показать график

plt.style.use("ggplot")  # готовая тема (seaborn-v0_8, bmh, dark_background)
plt.style.available      # список доступных тем
```

2.2 Типы графиков

```python
ax.plot(x, y)                                          # линия: динамика, тренды
ax.scatter(x, y, c=labels, s=20, cmap="viridis", alpha=0.7)  # точки: цвет=класс, размер=s
ax.bar(cats, vals)                                     # столбцы: сравнение категорий
ax.barh(cats, vals)                                    # горизонтальные столбцы (удобно для длинных
                                                       # названий)
ax.hist(data, bins=30, density=True, alpha=0.7)        # гистограмма распределения
ax.boxplot([d1, d2], labels=["a", "b"])                # ящик с усами: медиана, квартили, выбросы
ax.violinplot([d1, d2])                                # «скрипка»: форма распределения
ax.pie(vals, labels=names, autopct="%1.1f%%")          # круговая диаграмма с процентами
ax.errorbar(x, y, yerr=err, fmt="o")                   # точки с «усами» погрешности
ax.imshow(matrix, cmap="coolwarm", aspect="auto")      # матрица в виде цветной картинки
plt.colorbar(im)                                       # шкала цветов рядом с графиком
ax.contour(X, Y, Z)                                    # линии уровня функции двух переменных
ax.contourf(X, Y, Z, alpha=0.3)                        # то же, но с заливкой
ax.stackplot(x, y1, y2)                                # накопительные области
ax.step(x, y)                                          # ступенчатый график
ax.hexbin(x, y, gridsize=30)                           # плотность точек в шестиугольных ячейках
ax.hist2d(x, y, bins=30)                               # двумерная гистограмма
ax.stem(x, y)                                          # «леденцы»: вертикальные линии с маркерами
ax.quiver(X, Y, U, V)                                  # векторное поле (стрелки)
ax.twinx()                                             # вторая ось Y с общей осью X
```

2.3 Несколько графиков

```
fig, axes = plt.subplots(             # сетка графиков
    2, 3, figsize=(15, 8),            # 2 строки, 3 столбца, размер полотна
    sharex=True, sharey=False,        # общая ось X у всех, ось Y у каждого своя
)
for ax, col in zip(axes.flat, cols):  # обход графиков вместе со столбцами
    ax.hist(df[col])                  # гистограмма столбца
    ax.set_title(col)                 # заголовок = имя столбца

fig = plt.figure(figsize=(10, 6))  # пустое полотно
gs = fig.add_gridspec(2, 2)        # сетка 2x2 для произвольной раскладки
ax1 = fig.add_subplot(gs[0, :])    # график на всю верхнюю строку
ax2 = fig.add_subplot(gs[1, 0])    # график в левой нижней ячейке

fig, ax = plt.subplots(subplot_kw={"projection": "3d"})  # трёхмерный график
ax.scatter(x, y, z)                                      # точки в 3D
```

2.4 Графики прямо из pandas

```
df.plot(kind="line", figsize=(10, 4))                       # линии по всем числовым столбцам
df["a"].plot.hist(bins=30)                                  # гистограмма одного столбца
df.plot.scatter(x="a", y="b", c="target", cmap="coolwarm")  # точки, цвет по таргету
df.plot.box()                                               # ящики с усами для всех столбцов
df.plot.bar(stacked=True)                                   # столбцы с накоплением
df.plot.area()                                              # области с накоплением
df.hist(bins=30, figsize=(12, 10))                          # гистограммы ВСЕХ числовых столбцов сразу
pd.plotting.scatter_matrix(                                 # матрица парных диаграмм рассеяния
    df, figsize=(10, 10),                                   # данные и размер
    diagonal="kde",                                         # на диагонали рисовать плотность распределения
)
```

2.5 Визуализации для EDA и ML

```
# --- Матрица корреляций ---
corr = df.corr(numeric_only=True)                       # считаем корреляции
fig, ax = plt.subplots(figsize=(8, 6))                  # полотно
im = ax.imshow(corr, cmap="coolwarm", vmin=-1, vmax=1)  # цвет = сила связи от -1 до 1
ax.set_xticks(range(len(corr)))                         # позиции подписей по X
ax.set_xticklabels(corr.columns, rotation=90)           # имена признаков по X
ax.set_yticks(range(len(corr)))                         # позиции подписей по Y
ax.set_yticklabels(corr.columns)                        # имена признаков по Y
plt.colorbar(im)                                        # шкала цветов

# --- Распределение классов и пропусков ---
df["target"].value_counts().plot.bar()      # сколько объектов в каждом классе
df.isna().mean().sort_values().plot.barh()  # доля пропусков по столбцам

# --- Матрица ошибок ---
from sklearn.metrics import ConfusionMatrixDisplay  # отрисовка confusion matrix
ConfusionMatrixDisplay.from_estimator(              # строит матрицу по обученной модели
    model, X_test, y_test,                          # модель и тестовые данные
    normalize="true",                               # нормировать по строкам: в ячейках доли (recall)
    cmap="Blues",                                   # цветовая схема
)

# --- ROC и Precision-Recall ---
from sklearn.metrics import RocCurveDisplay, PrecisionRecallDisplay, DetCurveDisplay
RocCurveDisplay.from_estimator(model, X_test, y_test)         # ROC: TPR против FPR, площадь = AUC
PrecisionRecallDisplay.from_estimator(model, X_test, y_test)  # PR: лучше при дисбалансе

# --- Важность признаков ---
imp = pd.Series(                                  # собираем важности в Series
    model.feature_importances_, index=X.columns,  # значения и имена признаков
).sort_values()                                   # сортировка по возрастанию
imp.plot.barh()                                   # горизонтальные столбцы: сверху самые важные

# --- Кривые обучения ---
from sklearn.model_selection import LearningCurveDisplay, ValidationCurveDisplay
LearningCurveDisplay.from_estimator(        # качество от размера обучающей выборки
    model, X, y, cv=5, scoring="accuracy")  # показывает, нужно ли больше данных
ValidationCurveDisplay.from_estimator(      # качество от значения одного гиперпараметра
    model, X, y, param_name="max_depth",    # какой параметр меняем
    param_range=range(1, 15), cv=5,         # диапазон значений, число фолдов
)
```

```python
# --- Калибровка вероятностей ---
from sklearn.calibration import CalibrationDisplay
CalibrationDisplay.from_estimator(     # предсказанная вероятность против реальной частоты
    model, X_test, y_test, n_bins=10)  # идеал: точки на диагонали

# --- Граница решений (только 2 признака) ---
from sklearn.inspection import DecisionBoundaryDisplay
DecisionBoundaryDisplay.from_estimator(               # раскрашивает плоскость по предсказанию модели
    model, X2d, response_method="predict", alpha=0.4)  # X2d: матрица из двух признаков
plt.scatter(X2d[:, 0], X2d[:, 1], c=y, edgecolor="k")  # поверх рисуем точки данных

# --- Partial dependence ---
from sklearn.inspection import PartialDependenceDisplay
PartialDependenceDisplay.from_estimator(  # как изменение признака влияет на ответ
    model, X, features=[0, 1, (0, 1)])    # признаки 0 и 1 по отдельности и вместе

# --- Дерево решений ---
from sklearn.tree import plot_tree
plot_tree(                            # рисует обученное дерево
    tree, feature_names=X.columns,  # модель и имена признаков
    class_names=True, filled=True,  # названия классов, заливка по классу
    max_depth=3,                    # показать только 3 верхних уровня
)

# --- Проекция PCA / t-SNE ---
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y, cmap="tab10", s=10)  # данные в 2D, цвет = класс
```

Для статистической графики удобна надстройка **seaborn**: `sns.heatmap` (тепловая карта), `sns.pairplot` (парные графики), `sns.boxplot` (ящики), `sns.kdeplot` (плотность), `sns.countplot` (частоты категорий), `sns.clustermap` (кластеризованная тепловая карта).

3. Scikit-learn для классификации

Единый интерфейс всех моделей: **fit(X, y)** — обучить, **predict(X)** — предсказать класс, **predict_proba(X)** — вероятности классов, **score(X, y)** — качество. Преобразователи (скейлеры, кодировщики) работают через **fit / transform / fit_transform**.

3.1 Подготовка данных и разбиение

Таблица: что делает каждый класс/функция.

Что	За что отвечает
train_test_split	делит данные на обучающую и тестовую части
StratifiedKFold	k-фолд с сохранением долей классов в каждом фолде
KFold	обычный k-фолд (для регрессии или сбалансированных данных)
RepeatedStratifiedKFold	стратифицированный k-фолд, повторённый несколько раз (стабильнее оценка)
StratifiedShuffleSplit	случайные стратифицированные разбиения
LeaveOneOut	каждый объект по очереди становится тестом (очень долго)
GroupKFold	объекты одной группы (пациент, клиент) не попадают в train и test одновременно
TimeSeriesSplit	разбиение по времени: учимся на прошлом, проверяем на будущем
cross_val_score	оценка модели кросс-валидацией, возвращает массив метрик
cross_validate	то же, но несколько метрик, время и train-оценки
cross_val_predict	предсказания для каждого объекта, полученные «вне обучения»
GridSearchCV	полный перебор сетки гиперпараметров с кросс-валидацией
RandomizedSearchCV	случайный поиск гиперпараметров (быстрее на больших пространствах)
HalvingGridSearchCV	перебор с отсевом слабых вариантов на малых данных

```
from sklearn.model_selection import train_test_split  # функция разбиения на train/test

X_train, X_test, y_train, y_test = train_test_split(  # возвращает 4 части данных
    X, y,                                             # признаки и целевая переменная
    test_size=0.2,                                    # 20% данных уходит в тест
    stratify=y,                                       # сохранить доли классов в обеих частях
    random_state=42,                                  # фиксирует случайность: результат воспроизводим
)
```

3.2 Предобработка

Класс	За что отвечает
StandardScaler	приводит признак к среднему 0 и std 1 (z-оценка)
MinMaxScaler	сжимает признак в диапазон [0, 1]
RobustScaler	масштабирует по медиане и IQR, устойчив к выбросам
MaxAbsScaler	делит на максимум по модулю, сохраняет разреженность
Normalizer	нормирует каждую СТРОКУ к единичной длине
PowerTransformer	делает распределение ближе к нормальному (Yeo-Johnson, Box-Cox)
QuantileTransformer	переводит распределение в равномерное или нормальное по квантилям
OneHotEncoder	категория -> набор столбцов 0/1

Класс	За что отвечает
`OrdinalEncoder`	категория -> номер (для порядковых признаков и деревьев)
`TargetEncoder`	категория -> среднее таргета (встроена защита от утечки через кросс-фиттинг)
`LabelEncoder`	кодирует целевую переменную y в числа (НЕ для признаков)
`LabelBinarizer`	кодирует y в one-hot
`PolynomialFeatures`	добавляет степени и произведения признаков
`KBinsDiscretizer`	разбивает непрерывный признак на интервалы
`Binarizer`	значение выше порога -> 1, иначе 0
`FunctionTransformer`	превращает любую функцию в шаг пайплайна
`SimpleImputer`	заполняет пропуски (медиана, среднее, мода, константа)
`KNNImputer`	заполняет пропуски по значениям ближайших соседей
`IterativeImputer`	предсказывает пропуски другими признаками моделью
`make_pipeline`	цепочка шагов с автоматическими именами
`make_column_transformer`	разная обработка для разных групп столбцов
`make_column_selector`	выбирает столбцы по типу данных или по имени
`make_union`	склеивает результаты нескольких преобразователей рядом

```
from sklearn.preprocessing import StandardScaler, OneHotEncoder  # скейлер и кодировщик
from sklearn.impute import SimpleImputer                         # заполнение пропусков
from sklearn.compose import make_column_transformer, make_column_selector
from sklearn.pipeline import make_pipeline, make_union

num_pipe = make_pipeline(                                        # пайплайн для числовых признаков
    SimpleImputer(strategy="median"),                            # 1) пропуски -> медиана столбца
    StandardScaler(),                                            # 2) масштабирование (среднее 0, std 1)
)
cat_pipe = make_pipeline(                                        # пайплайн для категориальных признаков
    SimpleImputer(strategy="most_frequent"),                     # 1) пропуски -> самая частая категория
    OneHotEncoder(handle_unknown="ignore"),                      # 2) one-hot; новые категории в test не
                                                                 # ломают модель
)
prep = make_column_transformer(                                  # применяет разные пайплайны к разным
                                                                 # столбцам
    (num_pipe, make_column_selector(dtype_include="number")),  # числа -> num_pipe
    (cat_pipe, make_column_selector(dtype_include=["object", "category"])),  # текст -> cat_pipe
)
```

Имена шагов в `make_pipeline` генерируются автоматически: это имя класса в нижнем регистре (`simpleimputer`, `standardscaler`, `randomforestclassifier`). Через них обращаются к параметрам: `randomforestclassifier__max_depth`. Посмотреть имена можно так: `model.named_steps.keys()`. Если в пайплайне два шага одного класса, к имени добавляется номер (`standardscaler-1`, `standardscaler-2`).

Какой скейлер выбрать:

- **StandardScaler** — для линейных моделей, SVM, KNN, нейросетей.
- **RobustScaler** — когда в данных есть выбросы.
- **MinMaxScaler** — для нейросетей и алгоритмов, где нужен диапазон [0, 1].
- **PowerTransformer / QuantileTransformer** — для скошенных распределений.
- Деревьям и ансамблям на деревьях масштабирование не нужно.

3.3 Алгоритмы классификации

Линейные модели. Проводят разделяющую гиперплоскость. Быстрые и интерпретируемые, нужен скейлинг.

Класс	За что отвечает
LogisticRegression	вероятностная линейная модель, главный baseline
LogisticRegressionCV	логрегрессия с подбором C кросс-валидацией
RidgeClassifier	линейная модель с L2-регуляризацией (метод наименьших квадратов)
SGDClassifier	линейная модель, обучаемая стохастическим градиентным спуском (большие данные)
Perceptron	простейший линейный классификатор
PassiveAggressiveClassifier	онлайн-обучение, подходит для потоковых данных

```
from sklearn.linear_model import LogisticRegression, RidgeClassifier, SGDClassifier

LogisticRegression(
    C=1.0,                  # обратная сила регуляризации: меньше C = сильнее штраф, проще модель
    penalty="l2",           # тип штрафа: l2 (сглаживает), l1 (обнуляет признаки), elasticnet
    solver="lbfgs",         # алгоритм оптимизации (для l1/elasticnet нужен "saga")
    max_iter=1000,          # максимум итераций, увеличьте при ConvergenceWarning
    class_weight="balanced",  # веса классов обратно пропорциональны их частоте
)
RidgeClassifier(alpha=1.0)    # alpha: сила регуляризации (больше = проще модель)
SGDClassifier(
    loss="log_loss",        # функция потерь: log_loss = логрегрессия, hinge = SVM
    penalty="l2",           # тип регуляризации
    alpha=1e-4,             # сила регуляризации
)
```

Метод опорных векторов (SVM). Ищет границу с максимальным зазором между классами. Ядра позволяют строить нелинейные границы.

Класс	За что отвечает
SVC	SVM с любыми ядрами (rbf, linear, poly, sigmoid)
LinearSVC	линейный SVM, быстрее на больших данных
NuSVC	SVM с параметром nu вместо C (доля опорных векторов)

```
from sklearn.svm import SVC, LinearSVC

SVC(
    C=1.0,              # штраф за ошибки: больше C = жёстче граница, риск переобучения
    kernel="rbf",       # ядро: rbf (нелинейное), linear, poly, sigmoid
    gamma="scale",      # «радиус влияния» объекта для rbf: больше gamma = сложнее граница
    probability=True,   # включить predict_proba (медленнее: внутри кросс-валидация)
)
LinearSVC(
    C=1.0,              # штраф за ошибки
    dual="auto",        # выбор формы оптимизационной задачи автоматически
)
```

Ближайшие соседи. Класс определяется по классам ближайших объектов обучающей выборки.

Класс	За что отвечает
KNeighborsClassifier	голосование k ближайших соседей
RadiusNeighborsClassifier	голосование всех соседей в заданном радиусе
NearestCentroid	относит объект к классу с ближайшим центром (центроидом)

```
from sklearn.neighbors import KNeighborsClassifier

KNeighborsClassifier(
    n_neighbors=5,       # сколько соседей участвует в голосовании
```

```
    weights="distance",  # ближние соседи весомее ("uniform" = все равны)
    metric="minkowski",  # способ измерять расстояние
    p=2,                 # p=2 евклидово расстояние, p=1 манхэттенское
)
```

Наивный Байес. Применяет теорему Байеса, считая признаки независимыми. Очень быстрый.

Класс	Когда применять
GaussianNB	непрерывные признаки с нормальным распределением
MultinomialNB	счётчики, частоты слов (текст)
ComplementNB	текст при несбалансированных классах
BernoulliNB	бинарные признаки (слово есть / нет)
CategoricalNB	категориальные признаки

```
from sklearn.naive_bayes import GaussianNB, MultinomialNB

GaussianNB()              # без важных параметров, работает «из коробки»
MultinomialNB(alpha=1.0)  # alpha: сглаживание Лапласа, защита от нулевых вероятностей
```

Деревья и ансамбли. Деревья делят пространство признаков на области правилами «признак > порог». Ансамбли объединяют много деревьев.

Класс	За что отвечает
DecisionTreeClassifier	одно дерево решений, легко интерпретировать
ExtraTreeClassifier	дерево со случайными порогами разбиения
RandomForestClassifier	лес независимых деревьев на случайных подвыборках (bagging)
ExtraTreesClassifier	лес из деревьев со случайными порогами, быстрее и «шумнее»
GradientBoostingClassifier	деревья строятся последовательно, каждое исправляет ошибки прошлых
HistGradientBoostingClassifier	быстрый бустинг на гистограммах, понимает NaN и категории
AdaBoostClassifier	бустинг с повышением веса ошибочных объектов
BaggingClassifier	обучает любую модель на случайных подвыборках и усредняет
VotingClassifier	голосование разных моделей
StackingClassifier	метамодель учится на предсказаниях базовых моделей
IsolationForest	поиск аномалий (не классификатор в строгом смысле)

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import (RandomForestClassifier, GradientBoostingClassifier,
                              HistGradientBoostingClassifier, VotingClassifier,
                              StackingClassifier)

DecisionTreeClassifier(
    criterion="gini",                       # мера «нечистоты» узла: gini или entropy
    max_depth=5,                            # максимальная глубина дерева (главная защита от переобучения)
    min_samples_leaf=5,                     # минимум объектов в листе
    ccp_alpha=0.0,                          # сила обрезки дерева после обучения (больше = проще)
)
RandomForestClassifier(
    n_estimators=300,                       # число деревьев (больше = стабильнее, но дольше)
    max_depth=None,                         # глубина деревьев (None = без ограничения)
    max_features="sqrt",                    # сколько признаков рассматривать при каждом разбиении
    min_samples_leaf=1,                     # минимум объектов в листе (больше = менее переобучается)
    n_jobs=-1,                              # использовать все ядра процессора
    class_weight="balanced_subsample",      # балансировка классов в каждой подвыборке
    oob_score=True,                         # оценка качества на объектах, не попавших в подвыборку
    random_state=42,                        # воспроизводимость
)
GradientBoostingClassifier(
    n_estimators=200,                       # число последовательных деревьев
    learning_rate=0.1,                      # вклад каждого дерева (меньше = точнее, но нужно больше
                                            # деревьев)
```

```
    max_depth=3,                              # глубина слабых деревьев (обычно 3-5)
    subsample=0.8,                            # доля данных для каждого дерева (стохастический бустинг)
)
HistGradientBoostingClassifier(
    max_iter=300,                             # число деревьев
    learning_rate=0.1,                        # скорость обучения
    early_stopping=True,                      # остановка, когда качество на валидации перестало расти
    categorical_features="from_dtype",        # брать категориальные признаки из типа category
)
VotingClassifier(
    [("lr", lr), ("rf", rf), ("svc", svc)],   # список (имя, модель)
    voting="soft",                            # soft: усредняет вероятности, hard: голосует классами
)
StackingClassifier(
    estimators=[("rf", rf), ("svc", svc)],    # базовые модели
    final_estimator=LogisticRegression(),     # метамодель над их предсказаниями
    cv=5,                                     # предсказания базовых моделей берутся из кросс-валидации
)
```

Дискриминантный анализ, нейросети, прочее.

Класс	За что отвечает
LinearDiscriminantAnalysis	линейная граница в предположении нормальных классов с общей ковариацией
QuadraticDiscriminantAnalysis	то же, но у каждого класса своя ковариация (граница квадратичная)
MLPClassifier	полносвязная нейросеть
GaussianProcessClassifier	байесовская модель, даёт оценку неопределённости (малые данные)
DummyClassifier	«глупый» baseline: предсказывает самый частый класс или случайно
LabelPropagation	полуобучение: распространяет метки на неразмеченные данные
SelfTrainingClassifier	модель сама размечает уверенные неразмеченные объекты и доучивается

```
from sklearn.neural_network import MLPClassifier
from sklearn.dummy import DummyClassifier

MLPClassifier(
    hidden_layer_sizes=(128, 64),          # нейронов в скрытых слоях: 128, затем 64
    activation="relu",                     # функция активации нейронов
    solver="adam",                         # алгоритм оптимизации весов
    alpha=1e-4,                            # L2-регуляризация
    early_stopping=True,                   # остановить обучение, когда качество на валидации не растёт
    max_iter=500,                          # максимум эпох
)
DummyClassifier(strategy="most_frequent")  # baseline: всегда самый частый класс
```

Мультиклассовые и мультиметочные стратегии.

Класс	За что отвечает
OneVsRestClassifier	по одной бинарной модели «класс против всех»
OneVsOneClassifier	по модели на каждую пару классов
OutputCodeClassifier	кодирует классы бинарными кодами
MultiOutputClassifier	отдельная модель на каждую целевую переменную
ClassifierChain	цепочка моделей: каждая использует предсказания предыдущих (мультиметки)

```
from sklearn.multiclass import OneVsRestClassifier

OneVsRestClassifier(SVC())  # превратить бинарный SVM в мультиклассовый
```

Внешние библиотеки с sklearn-совместимым API: XGBoost (`XGBClassifier`), LightGBM (`LGBMClassifier`), CatBoost (`CatBoostClassifier`) — быстрые и точные градиентные бустинги.

3.4 Когда что выбирать

Алгоритм	Сильные стороны	Слабые стороны
Logistic Regression	Быстрый, интерпретируемый, хороший baseline	Только линейная граница
SVM (RBF)	Хорош на малых и средних данных, нелинейный	Медленный на больших данных, нужен скейлинг
KNN	Простой, без обучения	Медленное предсказание, страдает от высокой размерности
Naive Bayes	Очень быстрый, хорош для текста	Сильное допущение о независимости признаков
Decision Tree	Интерпретируемый, не требует скейлинга	Переобучается
Random Forest	Устойчив, мало настройки	Крупная модель, обычно слабее бустинга
Gradient Boosting	Обычно лучшее качество на табличных данных	Больше настройки, риск переобучения
MLP	Гибкий, ловит сложные зависимости	Нужен скейлинг и настройка

3.5 Обучение и предсказание

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import make_pipeline

model = make_pipeline(                                  # цепочка: предобработка + модель
    prep,                                               # шаг 1: ColumnTransformer из раздела 3.2
    RandomForestClassifier(random_state=42),            # шаг 2: классификатор
)
model.fit(X_train, y_train)                             # обучение (предобработка тоже обучается только на
                                                        # train)
y_pred = model.predict(X_test)                          # предсказанные классы
y_proba = model.predict_proba(X_test)                   # вероятности классов: [n_объектов, n_классов]
y_score = model.decision_function(X_test)               # «расстояние до границы» (SVM, линейные модели)
model.score(X_test, y_test)                             # accuracy на тесте
model.get_params()                                      # словарь всех параметров модели
model.set_params(randomforestclassifier__max_depth=5)   # изменить параметр шага пайплайна
model.named_steps                                       # доступ к шагам пайплайна по именам
```

3.6 Метрики классификации

Метрика	За что отвечает
accuracy_score	доля верных ответов (плоха при дисбалансе)
balanced_accuracy_score	средний recall по классам, честнее при дисбалансе
precision_score	точность: какая доля предсказанных «1» действительно «1»
recall_score	полнота: какую долю настоящих «1» нашли
f1_score	гармоническое среднее precision и recall
fbeta_score	F-мера с весом beta (beta>1 важнее recall)
roc_auc_score	вероятность, что случайная «1» получит оценку выше случайной «0»
average_precision_score	площадь под PR-кривой (PR-AUC), хороша при дисбалансе
log_loss	штраф за неуверенные и неверные вероятности
brier_score_loss	средний квадрат ошибки вероятностей
matthews_corrcoef	MCC: одна сбалансированная метрика от -1 до 1
cohen_kappa_score	согласие с эталоном с поправкой на случайность
jaccard_score	пересечение предсказаний и правды / их объединение

Метрика	За что отвечает
`hamming_loss`	доля неверно предсказанных меток
`confusion_matrix`	таблица «истинный класс × предсказанный класс»
`classification_report`	precision, recall, F1 по каждому классу
`roc_curve`	точки ROC-кривой
`precision_recall_curve`	точки PR-кривой для всех порогов
`top_k_accuracy_score`	верный ответ среди k самых вероятных классов
`make_scorer`	превращает свою метрику в scoring для GridSearchCV

```python
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             roc_auc_score, average_precision_score, confusion_matrix,
                             classification_report, matthews_corrcoef)

accuracy_score(y_test, y_pred)                    # доля правильных ответов
precision_score(y_test, y_pred, average="macro")  # precision, усреднение по классам поровну
recall_score(y_test, y_pred, average="macro")     # recall, усреднение по классам поровну
f1_score(y_test, y_pred, average="weighted")      # F1, классы взвешены по размеру
roc_auc_score(y_test, y_proba[:, 1])              # ROC-AUC бинарной задачи (вероятность класса 1)
roc_auc_score(y_test, y_proba, multi_class="ovr") # ROC-AUC мультикласса «один против всех»
average_precision_score(y_test, y_proba[:, 1])    # PR-AUC
confusion_matrix(y_test, y_pred)                  # матрица ошибок
print(classification_report(y_test, y_pred))      # подробный отчёт по классам
matthews_corrcoef(y_test, y_pred)                 # MCC
```

Параметр `average`: **binary** — только положительный класс, **macro** — среднее по классам (все равны), **weighted** — среднее с весом по размеру класса, **micro** — считать по всем объектам вместе.

Выбор метрики:

- **Сбалансированные классы:** accuracy, F1.
- **Дисбаланс:** PR-AUC, balanced accuracy, F1, MCC, ROC-AUC.
- **Дорогие пропуски (болезнь, фрод):** recall.
- **Дорогие ложные тревоги:** precision.
- **Важны вероятности:** log_loss, Brier score, калибровка.

3.7 Кросс-валидация

```python
from sklearn.model_selection import StratifiedKFold, cross_val_score, cross_validate, cross_val_predict

cv = StratifiedKFold(                # схема разбиения на фолды
    n_splits=5,                      # число фолдов
    shuffle=True,                    # перемешать данные перед разбиением
    random_state=42,                 # воспроизводимость
)
scores = cross_val_score(            # качество на каждом из 5 фолдов
    model, X, y,                     # модель (лучше целый пайплайн!) и данные
    cv=cv,                           # схема разбиения
    scoring="f1_macro",              # какую метрику считать
    n_jobs=-1,                       # параллельно на всех ядрах
)
print(scores.mean(), scores.std())   # средняя оценка и разброс между фолдами

res = cross_validate(                                # несколько метрик сразу
    model, X, y, cv=cv,                              # модель, данные, схема
    scoring=["accuracy", "f1_macro", "roc_auc_ovr"], # список метрик
    return_train_score=True,                         # заодно посчитать качество на train (видно
                                                     # переобучение)
)
y_cv_pred = cross_val_predict(                       # предсказание для каждого объекта «вне обучения»
    model, X, y, cv=cv,                              # модель, данные, схема
    method="predict_proba",                          # возвращать вероятности вместо классов
)
```

Для временных рядов используйте `TimeSeriesSplit`, при группированных данных (один пациент в нескольких строках) — `GroupKFold`.

3.8 Подбор гиперпараметров

```
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV
from scipy.stats import randint, uniform

param_grid = {                                                  # какие значения перебирать:
                                                                # «имя_шага__параметр»
    "randomforestclassifier__n_estimators": [100, 300, 500],    # число деревьев
    "randomforestclassifier__max_depth": [None, 5, 10],         # глубина деревьев
    "randomforestclassifier__min_samples_leaf": [1, 3, 5],      # минимум объектов в листе
}
gs = GridSearchCV(                                              # перебирает ВСЕ комбинации (здесь 3*3*3 =
                                                                # 27)
    model, param_grid,                                          # модель и сетка
    cv=cv,                                                      # кросс-валидация для каждой комбинации
    scoring="f1_macro",                                         # по какой метрике выбирать лучшую
    n_jobs=-1,                                                  # параллельно
    refit=True,                                                 # после поиска переобучить лучшую модель на
                                                                # всём train
)
gs.fit(X_train, y_train)                                        # запустить поиск
gs.best_params_                                                 # лучшая комбинация параметров
gs.best_score_                                                  # её средняя оценка на кросс-валидации
gs.best_estimator_                                              # готовая обученная лучшая модель
pd.DataFrame(gs.cv_results_)                                    # подробные результаты всех комбинаций

rs = RandomizedSearchCV(                                            # пробует N случайных комбинаций
    model,
    {"randomforestclassifier__max_depth": randint(3, 20),           # целое число из [3, 20)
     "randomforestclassifier__max_features": uniform(0.3, 0.7)},    # доля из [0.3, 1.0]
    n_iter=50,                                                      # сколько случайных комбинаций проверить
    cv=cv, scoring="roc_auc", n_jobs=-1, random_state=42,
)
```

Для крупных пространств есть `HalvingRandomSearchCV` (нужен `from sklearn.experimental import enable_halving_search_cv`), а также Optuna.

3.9 Дисбаланс классов

```
LogisticRegression(class_weight="balanced")                 # веса классов обратно пропорциональны частоте
RandomForestClassifier(class_weight="balanced_subsample")   # то же внутри каждой подвыборки
from sklearn.utils.class_weight import compute_class_weight, compute_sample_weight
model.fit(                                                  # обучение с весами объектов
    X, y,
    randomforestclassifier__sample_weight=compute_sample_weight("balanced", y),  # веса
)

# --- Подбор порога решения (по умолчанию порог 0.5) ---
from sklearn.model_selection import TunedThresholdClassifierCV  # sklearn >= 1.5
tuned = TunedThresholdClassifierCV(                             # подбирает порог вероятности под метрику
    model, scoring="f1", cv=5,                                  # метрика и число фолдов
).fit(X_train, y_train)                                         # обучение вместе с подбором порога

# --- Ресэмплинг (библиотека imbalanced-learn) ---
# SMOTE: создаёт синтетические объекты малого класса
# RandomUnderSampler: удаляет объекты большого класса
# SMOTEENN: SMOTE + очистка шума
# Используйте imblearn.pipeline.make_pipeline, чтобы ресэмплинг шёл только внутри
# обучающего фолда
```

3.10 Калибровка вероятностей

Калибровка нужна, когда важны сами вероятности, а не только порядок объектов: SVM, бустинг и лес часто выдают «кривые» вероятности.

```
from sklearn.calibration import CalibratedClassifierCV, calibration_curve

cal = CalibratedClassifierCV(  # оборачивает модель и исправляет её вероятности
    SVC(),                     # базовая модель
    method="isotonic",         # isotonic (гибко, нужно много данных) или sigmoid (проще)
    cv=5,                      # калибровка обучается на кросс-валидации
)
cal.fit(X_train, y_train)      # обучить модель и калибратор
```

3.11 Отбор и снижение размерности признаков

Класс	За что отвечает
VarianceThreshold	удаляет признаки с почти нулевой дисперсией
SelectKBest	оставляет k лучших признаков по статистическому тесту
SelectPercentile	оставляет заданный процент лучших признаков
f_classif	ANOVA F-тест (линейная связь с классом)
chi2	тест хи-квадрат (неотрицательные признаки)
mutual_info_classif	взаимная информация (ловит нелинейные связи)
RFE	рекурсивно отбрасывает самые слабые признаки
RFECV	RFE с автоматическим подбором числа признаков
SelectFromModel	оставляет признаки, важные по модели
SequentialFeatureSelector	жадно добавляет или удаляет признаки
PCA	главные компоненты: линейное сжатие с максимумом дисперсии
TruncatedSVD	аналог PCA для разреженных матриц (текст)
KernelPCA	нелинейный PCA через ядро
NMF	разложение на неотрицательные компоненты
FastICA	независимые компоненты
IncrementalPCA	PCA по частям для очень больших данных
TSNE	нелинейная проекция в 2D для визуализации кластеров
Isomap, MDS, LocallyLinearEmbedding, SpectralEmbedding	другие методы нелинейной проекции

```
from sklearn.feature_selection import (VarianceThreshold, SelectKBest, mutual_info_classif,
                                       RFECV, SelectFromModel, SequentialFeatureSelector)
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE

VarianceThreshold(threshold=0.01)                              # убрать признаки с дисперсией ниже 0.01
SelectKBest(score_func=mutual_info_classif, k=10)              # 10 лучших по взаимной информации
RFECV(                                                         # рекурсивный отбор с подбором числа
                                                               # признаков
    LogisticRegression(max_iter=1000),                         # модель, оценивающая важность
    cv=5, scoring="f1",                                        # кросс-валидация и метрика
)
SelectFromModel(RandomForestClassifier(), threshold="median")  # признаки важнее медианы
SequentialFeatureSelector(                                     # пошаговый отбор признаков
    KNeighborsClassifier(),                                    # модель для оценки
    n_features_to_select=5,                                    # сколько признаков оставить
    direction="forward",                                       # forward: добавляем, backward: удаляем
)
```

```
pca = PCA(n_components=0.95)          # столько компонент, чтобы сохранить 95% дисперсии
pca.explained_variance_ratio_         # доля дисперсии, объяснённая каждой компонентой
TSNE(n_components=2, perplexity=30)  # проекция в 2D (только для визуализации!)
```

3.12 Интерпретация моделей

```
model.feature_importances_  # важность признаков (деревья и ансамбли)
model.coef_                 # коэффициенты признаков (линейные модели)
model.intercept_            # свободный член (линейные модели)

from sklearn.inspection import permutation_importance
r = permutation_importance(                                    # важность = падение качества при
                                                               # перемешивании признака
    model, X_test, y_test,                                     # обученная модель и тестовые данные
    n_repeats=10,                                              # сколько раз перемешивать каждый признак
    scoring="f1_macro",                                        # метрика качества
    n_jobs=-1,                                                 # параллельно
)
pd.Series(r.importances_mean, index=X.columns).sort_values()  # средняя важность по признакам

from sklearn.tree import export_text
print(export_text(tree, feature_names=list(X.columns)))  # правила дерева в виде текста
```

Для SHAP и LIME используйте отдельные библиотеки `shap` и `lime`.

3.13 Текстовая классификация

Класс	За что отвечает
`CountVectorizer`	превращает тексты в матрицу счётчиков слов (bag of words)
`TfidfVectorizer`	то же, но слова взвешиваются по редкости (TF-IDF)
`HashingVectorizer`	признаки через хеширование: без словаря, экономит память

```
from sklearn.feature_extraction.text import TfidfVectorizer

text_clf = make_pipeline(               # векторизация + классификатор
    TfidfVectorizer(
        ngram_range=(1, 2),             # учитывать слова и пары слов
        min_df=3,                       # отбросить слова, встречающиеся менее чем в 3 документах
        max_df=0.9,                     # отбросить слова из более чем 90% документов (стоп-слова)
        sublinear_tf=True,              # логарифмировать частоту слова
    ),
    LogisticRegression(max_iter=1000),  # линейная модель по TF-IDF
)
```

3.14 Сохранение и загрузка модели

```
import joblib                       # быстрая сериализация объектов sklearn
joblib.dump(model, "model.joblib")  # сохранить обученный пайплайн в файл
model = joblib.load("model.joblib")  # загрузить обратно и сразу предсказывать
```

3.15 Вспомогательные утилиты

```
from sklearn.utils import shuffle, resample  # перемешивание; ресэмплинг с возвратом
from sklearn.base import BaseEstimator, TransformerMixin, ClassifierMixin, clone
clone(model)                                 # новая необученная копия модели с теми же параметрами
from sklearn import set_config
set_config(display="diagram")                # красивое отображение пайплайна в Jupyter
set_config(transform_output="pandas")        # преобразователи возвращают DataFrame
from sklearn.datasets import load_iris, load_breast_cancer, make_classification
X, y = load_iris(return_X_y=True)            # учебный датасет «ирисы»
X, y = make_classification(n_samples=1000, n_features=20, random_state=42)  # синтетические данные
```

Назначение базовых классов: `BaseEstimator` — основа любой модели (get_params/set_params), `TransformerMixin` — добавляет `fit_transform`, `ClassifierMixin` — добавляет `score` для классификаторов. Через них пишутся собственные модели и шаги пайплайна.

4. Полный пример: от данных до модели

```
import pandas as pd, numpy as np, matplotlib.pyplot as plt  # данные, числа, графики
from sklearn.model_selection import train_test_split, StratifiedKFold, GridSearchCV
from sklearn.compose import make_column_transformer, make_column_selector
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, ConfusionMatrixDisplay, RocCurveDisplay

# 1. Данные
df = pd.read_csv("data.csv")                    # загрузка
df = df.drop_duplicates()                       # убираем повторяющиеся строки
X, y = df.drop(columns="target"), df["target"]  # признаки и целевая переменная

# 2. EDA (быстрый осмотр)
print(df.info())                            # типы и число непустых значений
print(df.isna().mean())                     # доли пропусков по столбцам
y.value_counts(normalize=True).plot.bar()  # баланс классов
plt.show()                                  # показать график

# 3. Разбиение
X_tr, X_te, y_tr, y_te = train_test_split(  # делим на train и test
    X, y, test_size=0.2,                    # 20% в тест
    stratify=y, random_state=42,            # сохраняем доли классов, фиксируем случайность
)

# 4. Пайплайн
prep = make_column_transformer(                             # разная обработка разных столбцов
    (make_pipeline(SimpleImputer(strategy="median"), StandardScaler()),  # числа: медиана + скейлер
     make_column_selector(dtype_include="number")),
    (make_pipeline(SimpleImputer(strategy="most_frequent"),  # текст: мода + one-hot
                   OneHotEncoder(handle_unknown="ignore")),
     make_column_selector(dtype_include="object")),
)
pipe = make_pipeline(prep, RandomForestClassifier(random_state=42, n_jobs=-1))  # предобработка + модель

# 5. Подбор гиперпараметров
cv = StratifiedKFold(5, shuffle=True, random_state=42)      # 5 стратифицированных фолдов
gs = GridSearchCV(                                          # перебор параметров по кросс-валидации
    pipe,
    {"randomforestclassifier__n_estimators": [200, 400],    # число деревьев
     "randomforestclassifier__max_depth": [None, 10, 20]},  # глубина деревьев
    cv=cv, scoring="f1_macro", n_jobs=-1,                   # схема, метрика, параллельность
)
gs.fit(X_tr, y_tr)                                          # поиск лучшей комбинации (только на train!)
print(gs.best_params_, gs.best_score_)                      # лучшие параметры и оценка

# 6. Оценка на тесте (один раз, в самом конце)
best = gs.best_estimator_                               # лучшая обученная модель
print(classification_report(y_te, best.predict(X_te)))  # precision/recall/F1 по классам

fig, axes = plt.subplots(1, 2, figsize=(12, 4))              # два графика рядом
ConfusionMatrixDisplay.from_estimator(best, X_te, y_te, ax=axes[0], cmap="Blues")  # матрица ошибок
RocCurveDisplay.from_estimator(best, X_te, y_te, ax=axes[1])  # ROC-кривая
plt.tight_layout()                                           # без наложений
plt.show()                                                   # показать
```

5. Частые ошибки

1. **Утечка данных:** скейлинг, импутация и кодирование нужно обучать только на train, поэтому всегда используйте `make_pipeline`.
2. **Нестратифицированное разбиение** при несбалансированных классах.
3. **Оценка только по accuracy** при дисбалансе.
4. **Подбор гиперпараметров на тесте:** тест трогаем один раз в самом конце.
5. **Игнорирование baseline:** сравнивайте с `DummyClassifier` и логистической регрессией.
6. **Нет `random_state`:** результаты нельзя воспроизвести.
7. **Использование `LabelEncoder` для признаков:** он предназначен для целевой переменной, для признаков берите `OrdinalEncoder` или `OneHotEncoder`.
8. **Лишний `inplace=True`:** в современном pandas он мало что даёт и осложняет цепочки вызовов.